

2주차 읽기자료: 정상상태 MLP 학습과 평가

1. 하나의 반응기, 세 정상상태 출력

1차 연속반응 $A \rightarrow B \rightarrow C$ 에서 각 시간 미분을 0으로 놓으면 A, B, C 순서로 정상 농도를 구할 수 있다.

$$\begin{aligned}C_A &= \frac{C_{Af}}{1 + k_1\tau} \\C_B &= \frac{k_1\tau C_{Af}}{(1 + k_1\tau)(1 + k_2\tau)} \\C_C &= k_2\tau C_B\end{aligned}$$

330 K에서 첫 번째와 두 번째 반응의 속도상수는 0.2/min, 0.1/min이다. 체류시간 5 min·유입 농도 1.2 mol/L이면 세 농도는 **[0.6, 0.4, 0.2] mol/L**다. 합은 1.2다. 정상상태 학습 행 하나의 label이 된다.

입력은 $[T, \tau, C_{Af}]$, 출력은 $[C_A, C_B, C_C]$ 다. 한 행은 하나의 정상상태 운전 조건이다.

2. 학습 전에 파라미터 수 계산하기

Dense layer는 입력-출력 쌍마다 weight 하나, 출력마다 bias 하나를 갖는다. 정상상태 MLP는 $3 \rightarrow 32 \rightarrow 32 \rightarrow 3$ 이다.

Layer	개수
첫 hidden layer	$3 \times 32 + 32 = 128$
두 번째 hidden layer	$32 \times 32 + 32 = 1056$
Output layer	$32 \times 3 + 3 = 99$
합계	1283

$$h_1 = \text{ReLU}(W_1 \tilde{x} + b_1)$$

$$h_2 = \text{ReLU}(W_2 h_1 + b_2)$$

$$\hat{z} = W_3 h_2 + b_3$$

ReLU는 hidden pre-activation마다 $\max(0, \text{값})$ 을 적용한다. Linear output은 음수 농도도 반환할 수 있으므로, 비음성과 농도 합의 일관성을 따로 평가해야 한다.

3. Scaling은 loss의 강조점을 바꾼다

입력 scaling에는 명시한 bounds를 사용한다. 출력 표준화의 평균·표준편차는 train 데이터에서만 계산한다. 농도 오차를 보고할 때는 물리 단위로 되돌린다.

$$\tilde{x}_j = 2 \frac{x_j - l_j}{u_j - l_j} - 1$$
$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

$$L = \frac{1}{3N} \sum_{n=1}^N \sum_{j=1}^3 \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

Raw squared error에는 train 표준편차 제공의 역수가 가중치로 붙는다. A의 표준편차가 약 0.283, B가 0.110이면, 같은 raw squared error에 B는 A보다 약 **6.62배**의 가중치를 받는다.

이는 가중치 선택의 의미이며 더 좋은 예측을 보장하지 않는다. 성분별 test RMSE와 농도 합 residual을 비교한다. [PyTorch 공식 MSE 정의](#)도 확인할 수 있다.

4. Backward 한 번, 파라미터 갱신 한 번

$3 \rightarrow 2 \rightarrow 3$ network의 초기값을 $W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, bias는 모두 0으로 둔다. 입력 $[1, 0, -1]$ 에 대한 hidden pre-activation은 $[1, -1]$, ReLU 출력은 $[1, 0]$, 예측은 $[1, 0, 1]$ 이다. Target $[0, 1, 0]$ 에 대한 오차는 $[1, -1, 1]$ 이다.

$$\begin{aligned}\delta_2 &= e, \nabla_{W_2} L = e h^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix} \\ \delta_1 &= \left(W_2^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \\ \nabla_{W_1} L &= \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}\end{aligned}$$

Output weight의 gradient는 오차 $\times$ hidden activation의 전치다. Hidden 오차에는 output weight와 ReLU 미분이 들어간다. 음수 pre-activation에서는 그 성분의 gradient가 막힌다.

Bias gradient는 해당 오차 벡터다. 학습률 0.1로 **weight와 bias를 모두 갱신**하면 다음 예측은 **[0.26, 0.14, 0.26]**이다. Loss $\frac{1}{2} \|\text{예측} - \text{target}\|^2$ 는 **1.5에서 0.4374로** 줄어든다.

[PyTorch autograd](#)가 이 chain rule 계산을 자동 수행한다. Notebook에서 손 계산을 확인한다.

5. 학습 loop를 계산 순서로 읽기

한 batch에서는 예측과 loss를 계산하고, 이전에 누적된 parameter gradient를 지운 뒤, 현재 loss를 미분하고 optimizer 갱신을 적용한다.

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

`loss.backward()`는 gradient를 계산하고 `optimizer.step()`은 파라미터를 바꾼다. Gradient를 지우지 않으면 다음 backward 계산이 기존 gradient에 누적된다.

Epoch 뒤 validation loss를 평가하고 최솟값의 weight를 보관한다. Test는 파라미터 학습과 모델 선택에 사용하지 않는다. 예측 지표는 출력 표준화를 되돌린 뒤 계산한다.

원문 해설: PyTorch [학습 loop](#), [모델 구성](#).

6. 독립 예측을 진단하기

기준값이 $[0.6, 0.4, 0.2]$ mol/L이고 predictor가 $[0.62, 0.39, 0.19]$ mol/L를 반환한다고 하자.

성분별 오차는 $[0.02, -0.01, -0.01]$ mol/L다. 두 농도 합은 모두 1.2이므로 농도 합 residual은 0이다. 전체 농도가 맞아도 각 성분이 정확한 것은 아니다.

확인: Test 보고서에 어떤 지표가 필요한가? 모델을 선택할 때 어떤 데이터를 사용해야 하는가?

확인 답: 물리 단위의 A·B·C RMSE, 농도 합 residual, 최소 예측 농도를 보고한다. Train으로 학습하고 validation으로 선택하며, test는 두 단계에 사용하지 않는다.

2주차 필수 실습으로 돌아간다.